```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----------------------------------
# USER SETTINGS
# -----------------------------------
log_file = "log.lammps"
temp_file = "temp.profile.direct"

# Number of final temperature-profile blocks to average
n_last_blocks = 3

# Choose the linear region for fitting
fit_chunk_min = 22
fit_chunk_max = 32

# Use later part of the log file for fitting thermostat energy
log_fit_start = 40000

# Conversion factor:
# 1 eV / (ps·Angstrom·K) = 1602.176634 W/m·K
EV_PER_PS_A_TO_WMK = 1602.176634

# -----------------------------------
# PART 1: READ TEMPERATURE PROFILE
# -----------------------------------
with open(temp_file, "r") as f:
    lines = f.readlines()

blocks = {}
i = 0

while i < len(lines):
    line = lines[i].strip()

    if not line or line.startswith("#"):
        i += 1
        continue

    parts = line.split()

    if len(parts) == 3:
        try:
            ts = int(float(parts[0]))
            nchunk = int(float(parts[1]))
            _ = float(parts[2])
        except ValueError:
            i += 1
            continue
```

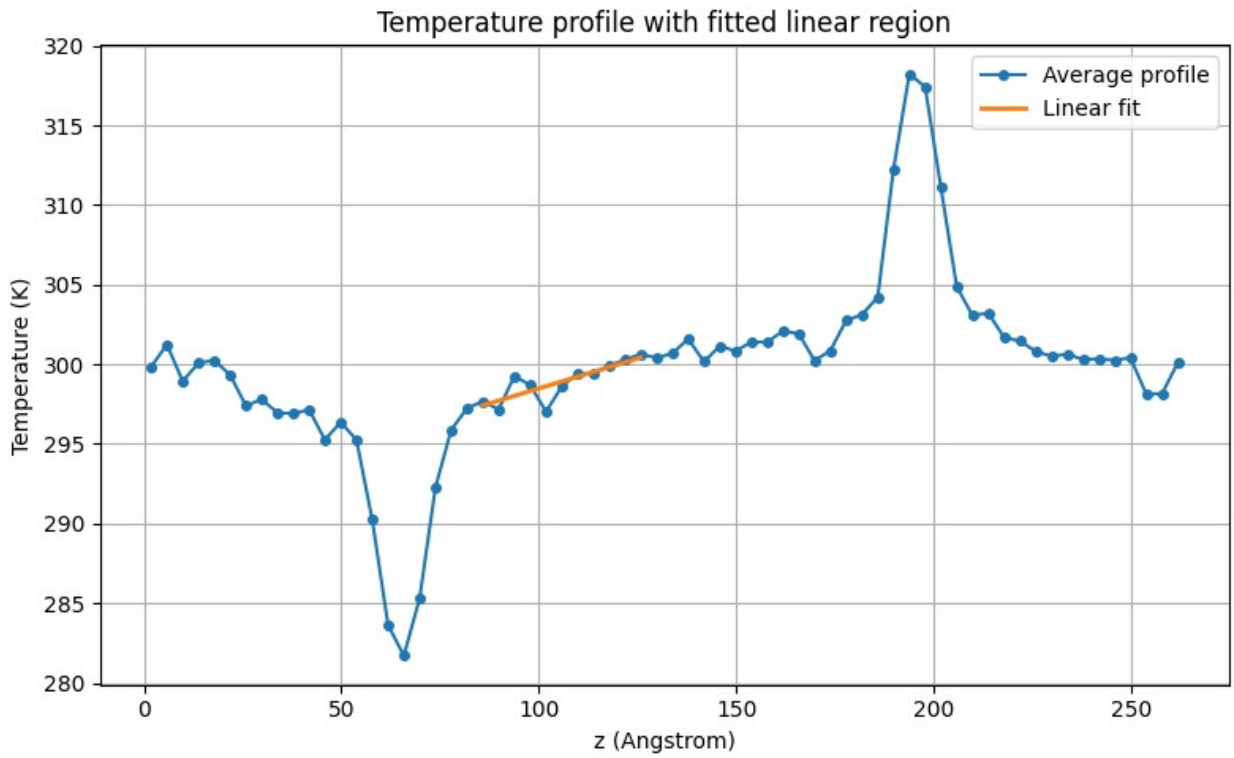
```
        data = []
        for _ in range(nchunk):
            i += 1
            p = lines[i].split()
            data.append([
                int(p[0]),      # chunk number
                float(p[1]),    # z coordinate
                float(p[2]),    # atom count
                float(p[3])     # temperature
            ])

        blocks[ts] = pd.DataFrame(
            data, columns=["chunk", "coord", "ncount", "temp"]
        )

    i += 1

all_steps = sorted(blocks.keys())
selected_steps = all_steps[-n_last_blocks:]

df_all = pd.concat(
    [blocks[ts].assign(timestep=ts) for ts in selected_steps],
    ignore_index=True
)

avg_profile = (
    df_all.groupby("chunk", as_index=False)
    .agg(coord=("coord", "mean"),
         ncount=("ncount", "mean"),
         temp=("temp", "mean"))
)

# -----------------------------------
# PART 2: FIT TEMPERATURE GRADIENT
# -----------------------------------
fit_df = avg_profile[
    (avg_profile["chunk"] >= fit_chunk_min) &
    (avg_profile["chunk"] <= fit_chunk_max)
].copy()

x = fit_df["coord"].values
y = fit_df["temp"].values

# Linear fit: y = m*x + b
m, b = np.polyfit(x, y, 1)
grad = m   # dT/dz in K/Angstrom

print("Temperature profile blocks used:", selected_steps)
print(f"Fitted chunk range: {fit_chunk_min} to {fit_chunk_max}")
```

```
print("Temperature gradient dT/dz =", grad, "K/Angstrom")

plt.figure(figsize=(8, 5))
plt.plot(avg_profile["coord"], avg_profile["temp"], "o-", ms=4,
label="Average profile")
plt.plot(x, m*x + b, "-", lw=2, label="Linear fit")
plt.xlabel("z (Angstrom)")
plt.ylabel("Temperature (K)")
plt.title("Temperature profile with fitted linear region")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# -----------------------------------
# PART 3: READ LOG FILE
# -----------------------------------
rows = []
capture = False

with open(log_file, "r") as f:
    for line in f:
        s = line.strip()

        # Start reading after thermo header
        if s.startswith("Step") and ("f_HOT" in s) and ("f_COLD" in
s):
            capture = True
            continue

        if capture:
            if s.startswith("Loop time"):
                break

            vals = s.split()
            if len(vals) == 11:
                try:
                    row = [int(vals[0])] + [float(v) for v in
vals[1:]]
                    rows.append(row)
                except ValueError:
                    pass

# Put log data into a DataFrame
log_df = pd.DataFrame(
    rows,
    columns=["step", "temp", "pe", "ke", "etotal", "press",
             "lx", "ly", "lz", "f_hot", "f_cold"]
)
```

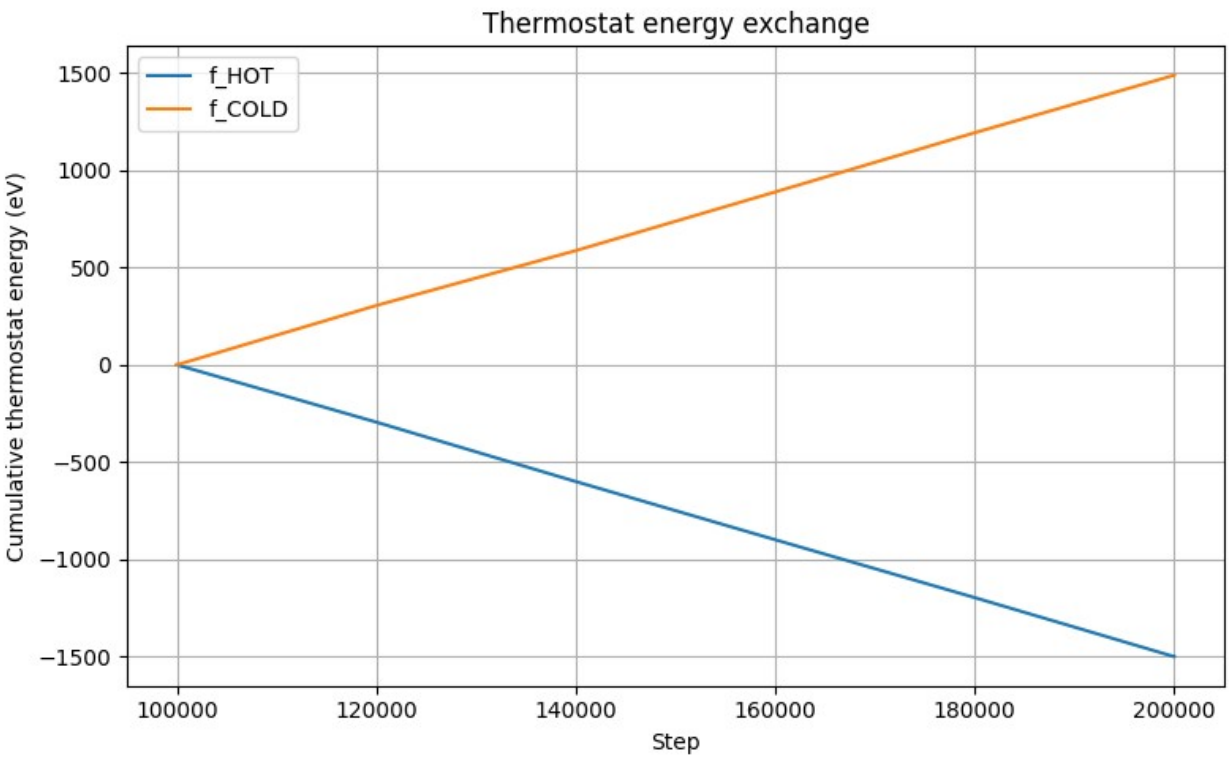
```python
# ------------------------------------
# PART 4: FIT THERMOSTAT ENERGY SLOPES
# ------------------------------------
log_fit_df = log_df[log_df["step"] >= log_fit_start].copy()

m_hot, b_hot = np.polyfit(log_fit_df["step"], log_fit_df["f_hot"], 1)
m_cold, b_cold = np.polyfit(log_fit_df["step"], log_fit_df["f_cold"],
1)

# Average of absolute slopes (energy per step)
dE_dstep = 0.5 * (abs(m_hot) + abs(m_cold))

# Timestep from LAMMPS input (metal units: ps)
timestep = 0.001

# Convert from energy/step to energy/time
Qdot = dE_dstep / timestep

print("Slope of f_HOT =", m_hot, "eV/step")
print("Slope of f_COLD =", m_cold, "eV/step")
print("Average energy rate =", dE_dstep, "eV/step")
print("Heat input rate Qdot =", Qdot, "eV/ps")

plt.figure(figsize=(8, 5))
plt.plot(log_df["step"], log_df["f_hot"], label="f_HOT")
plt.plot(log_df["step"], log_df["f_cold"], label="f_COLD")
plt.xlabel("Step")
plt.ylabel("Cumulative thermostat energy (eV)")
plt.title("Thermostat energy exchange")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# ------------------------------------
# PART 5: CALCULATE THERMAL CONDUCTIVITY
# ------------------------------------
Lx = log_df["lx"].iloc[-1]
Ly = log_df["ly"].iloc[-1]
A = Lx * Ly   # cross-sectional area in Angstrom^2

# Factor of 2 because heat flows in two directions in periodic setup
J = Qdot / (2.0 * A)

k_metal = J / abs(grad)
k_WmK = k_metal * EV_PER_PS_A_TO_WMK

print("Cross-sectional area A =", A, "Angstrom^2")
print("Heat flux J =", J, "eV/(ps*Angstrom^2)")
```

```
print("Thermal conductivity k =", k_metal, "eV/(ps*Angstrom*K)")
print("Thermal conductivity k =", k_WmK, "W/m-K")
```

```
Temperature profile blocks used: [390000, 420000, 450000]
Fitted chunk range: 22 to 32
Temperature gradient dT/dz = 0.0752669816324602 K/Angstrom
```




```
Slope of f_HOT = -0.015025189157414999 eV/step
Slope of f_COLD = 0.014891031554056888 eV/step
Average energy rate = 0.014958110355735944 eV/step
Heat input rate Qdot = 14.958110355735943 eV/ps
```




```
Cross-sectional area A = 2957.1779831634813 Angstrom^2
Heat flux J = 0.0025291190521671445 eV/(ps*Angstrom^2)
Thermal conductivity k = 0.03360197256902378 eV/(ps*Angstrom*K)
Thermal conductivity k = 53.83629530639885 W/m-K
```